Библиотека для экспериментов с временными рядами

episynth

Proof of Concept

Prequential-оценка эпиплексии:
теория, архитектура, реализация

Алексей Кравацкий · Марк Иконников
Даниил Казачков · Данила Черноусов

github.com/intsystems/Epiplexity

21 октября 2026

Что проверяем в исследовании

Вопрос

Связано ли количество структуры, извлечённой из синтетического генератора, с качеством прогноза на реальных временных рядах?

Для каждого генератора g

- обучаем Moirai с нуля;
- считаем оценку $\widehat{S}_g$;
- проверяем перенос на GIFT-Eval.

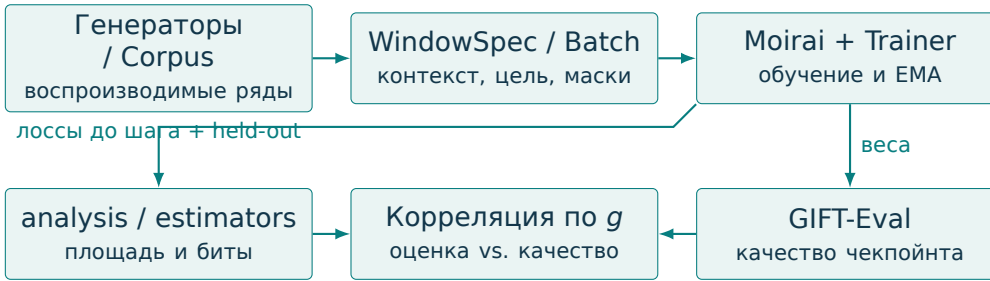
Условия сравнения

- одна архитектура;
- общий бюджет в серии;
- 11 генераторов / 55 задач.

$$g \longmapsto (\widehat{S}_g, Q_g), \quad \rho = \mathrm{Spearman}(\widehat{S}_g, Q_g).$$

Q_g — ошибка прогноза: меньше — лучше. Ожидаемый знак полезной связи — отрицательный.

Архитектура следует структуре эксперимента



- `train.py`: читает JSON, выбирает компоненты и запускает оценщик.
- Общие слои не зависят от выбора алгоритма оценивания.
- `estimators/`: единый контракт и реестр расширений.

Prequential: предсказать → обучить

Для окна i: контекст C_i, будущее Y_i, номер батча $b(i)$.

$$\ell_i = -\log q_{\theta_{b(i)-1}}(Y_i \mid C_i), \qquad L_{\text{pre}} = \sum_{i=1}^{N} \ell_i.$$

```
for batch in stream:
    per_window, _ = self.compute_loss(batch)
    loss = per_window.mean() / self.spec.horizon
    self.optimizer.zero_grad(set_to_none=True)
    loss.backward()
    self.optimizer.step()
    # persist per_window: pre-update values
```

Блочный prequential: окна батча оцениваются общей моделью. Запись лоссов после шага не меняет их предобновлённые значения. Clipping и EMA опущены.

Ключевое условие: оценка окна до обучения на соответствующем батче.

От prequential-лосса к эпиплексии

Prequential-лосс включает ошибки до освоения закономерностей. Из него вычитаем финальный held-out уровень за тот же бюджет.

$$\bar{\ell}_{\text{fin}} = \frac{1}{m} \sum_{j=1}^{m} -\log q_{\bar{\theta}_{\text{fin}}}(Y_j^{\text{held}} \mid C_j^{\text{held}})$$

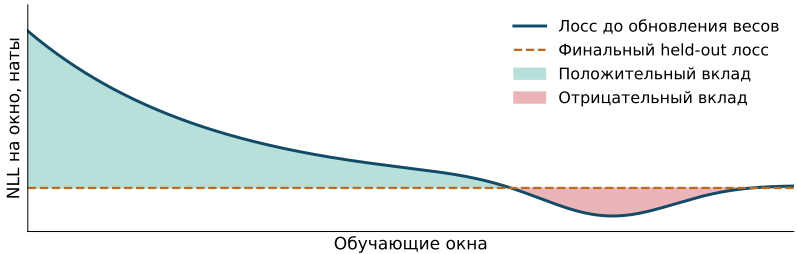
Разложение, используемое в episynth

$$\widehat{H} = \frac{N\bar{\ell}_{\text{fin}}}{\ln 2}, \quad \widehat{S} = \frac{L_{\text{pre}} - N\bar{\ell}_{\text{fin}}}{\ln 2}, \quad \widehat{S} + \widehat{H} = \frac{L_{\text{pre}}}{\ln 2}.$$

$\bar{\theta}_{\text{fin}}$ — финальные EMA-веса, m — число независимых от обучения held-out окон. Лоссы в натах, результат в битах.

Эмпирическая оценка, а не точная теоретическая эпиплексия.

Геометрический смысл: знаковая площадь



$$A = \sum_i (\ell_i - \bar{\ell}_{\text{fin}}), \qquad \widehat{S} = A / \ln 2.$$

Схематическая иллюстрация, не измеренный прогон. Участки ниже baseline дают отрицательный вклад; код не обрезает площадь снизу нулём.

Как формула реализована в коде

```
# PrequentialEstimator.run(...)
Trainer(model, generator, spec, config, out).run()
return self.from_run_dir(out)

# analysis.run_area(...): simplified
L_pre = losses.sum()
l_fin = heldout.mean()
A = L_pre − len(losses) ∗ l_fin
```

`window_loss.f32`	Лосс каждого окна до обновления весов.
`heldout_ema_1.f32`	Held-out лоссы конечной EMA-модели.
`PrequentialEstimator`	Запускает обучение и переводит результат в биты.

Исходники: `estimators/prequential.py`, `analysis.py`, `training.py`.

Конфигурация маленького CPU-прогона

Данные и форма задачи

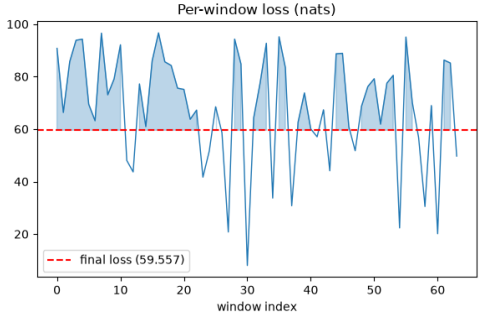
- `waveform`, длина ряда 256;
- контекст 128, горизонт 32;
- размер патча 32;
- 64 train-окна, 32 held-out;
- батч 4: всего 16 шагов.

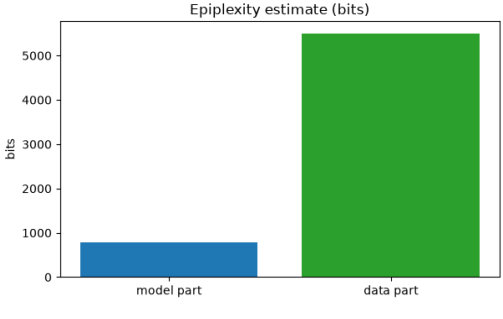
Модель и обучение

- ширина 64, один слой;
- 60 994 параметра;
- распределение Student-t;
- CPU, fp32, init seed 0;
- AdamW, EMA, раздельные сиды.

Связь с большими экспериментами: те же классы, реестр и конфигурационные объекты, что и в GPU-прогонах; уменьшен вычислительный бюджет.

PoC выполнен: лоссы, held-out и оценка в битах





Model part	Data part	Total
788.284 бит	5499.059 бит	6287.343 бит

График из выполненного demo_estimators.ipynb: 6 кодовых ячеек, ошибок выполнения нет. Заливка выделяет только участки выше baseline; численный расчёт A учитывает оба знака. Короткий прогон не доказывает сходимость оценки или корреляцию с переносом.

Интеграция алгоритмов через один контракт

```python
class EpiplexityEstimator(Protocol):
    name: str
    def run(
        self, model: MoiraiModule, generator: Generator,
        spec: WindowSpec, config: TrainConfig,
        out_dir: str | Path,
    ) -> dict[str, float]: ...
```

1. Реализовать `run`: вернуть `model_part_bits` и `data_part_bits`.
2. Зарегистрировать класс: `@register("name")`.
3. Выбрать через `build("name")` или поле JSON `estimator`.

Prequential реализован. Requential — каркас, вызывающий NotImplementedError. В ноутбуке ConstantEstimator показывает регистрацию, а не научный алгоритм.

Автодок и проверки реализации

Автодок Sphinx

```
.. automodule:: episynth.analysis
   :members:
```

Сигнатуры, аннотации и docstrings автоматически превращаются в API-страницу.
Открыть автодок run_area

Что проверяется

- pytest: формулы площади, обучение, файлы и resume;
- генерация, окна и модель;
- Ruff: стиль и импорты;
- Sphinx: предупреждения считаются ошибками сборки.

Последняя полная проверка базового кода: 72 passed, 2 skipped. Пропуски требуют внешних данных GIFT-Eval и весов Moirai. Сборка docs на push/PR, публикация Pages только вручную.

Что подтверждает Proof of Concept

1. **Ноутбук:** конфигурация → `build("prequential")` → прогон → графики и биты.
2. **Код:** контракт → PrequentialEstimator → общий Trainer → `run_area`.
3. **Документация:** сгенерированная API-страница с параметрами и результатом функции.

Подтверждённый результат

Базовый алгоритм встроен в библиотеку: данные проходят весь путь от генератора до сохранённых лоссов и оценки. Интерфейс позволяет подключать новые оценщики.

Выполненный ноутбук Документация Технический отчёт

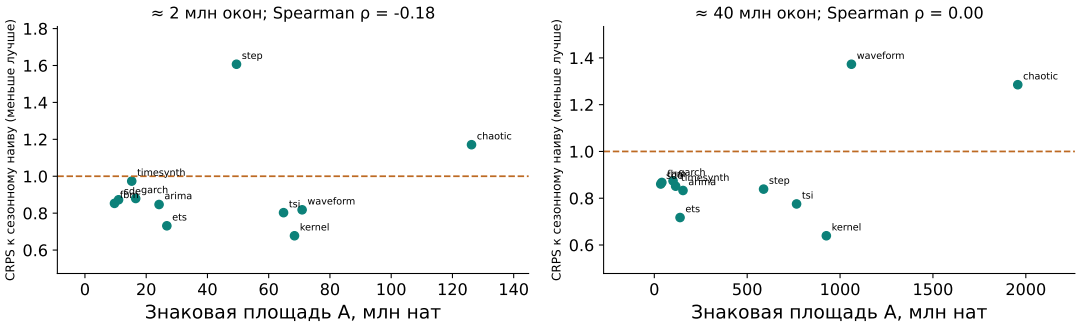
Приложение: неопределённость и ограничения

$$\mathrm{SE}(\widehat{S}) = \frac{N}{\ln 2}\,\mathrm{SE}(\bar{\ell}_{\mathrm{fin}}), \qquad \mathrm{SE}(\bar{\ell}_{\mathrm{fin}}) = \frac{s_{\mathrm{held}}}{\sqrt{m}}.$$

- Это ошибка от конечного held-out набора при независимости окон; она не включает разброс по сидам обучения.
- EMA и raw endpoint могут давать разные площади. Фиксируем протокол и проверяем чувствительность.
- Для непрерывных плотностей NLL и разложение могут быть отрицательными; важны масштаб и деквантование.
- Один прогон не решает теоретическую оптимизацию по наблюдателям: оценка привязана к архитектуре и бюджету.

Отрицательная знаковая площадь сама по себе не подтверждает отрицательную теоретическую эпиплексию.

Приложение: контекст больших серий



Источник: сохранённые CSV техотчёта, по 11 генераторов. Отдельный повтор Waveform по сиду исключён. Штриховая линия — качество сезонного наива.

Во всех 11 семействах связь площади с агрегированным CRPS слабая: $\rho \approx -0.18$ и 0.00; bootstrap-интервалы включают ноль. Это результат большого исследования, не маленького PoC.

Источники и воспроизводимость

- Finzi et al. (2026), *From Entropy to Epiplexity*. arXiv:2601.03220. Теория и prequential-подход.
- Woo et al. (2024), *Unified Training of Universal Time Series Forecasting Transformers*. arXiv:2402.02592. Moirai.
- Aksu et al. (2024), *GIFT-Eval*. arXiv:2410.10393. Бенчмарк переноса.
- Nagpal et al. (2026), *Mix, Don't Pick*. arXiv:2606.09912. Генераторы.

Данные: выполненный ноутбук и CSV в `publications/technical_report/data/`.

Базовая ревизия: `c910088`. Материалы презентации: `publications/slides/poc_prequential/`.

Схема площади отделена от измеренных данных. Большие GPU-эксперименты при подготовке не перезапускались.